

Documento de Prueba Unitaria — Límite de Envío de Códigos de Recuperación (xUnit)

Proyecto: ANPR-VISION (Backend)

Módulo: Business → `UserBusiness.RequestPasswordResetAsync`

Versión del documento: 1.0

Autor: Karol Natalia Osorio Poveda

Fecha de ejecución: 08/09/2025

Duración estimada: 10–15 min

Tipo de prueba: Unitaria (con dobles de prueba / mocks)

Herramientas: .NET SDK (9), xUnit, Moq, FluentAssertions, Test Explorer (VS Code)

Repositorio/Ubicación de pruebas:

`tests/AnprVision.Business.Tests/UserBusiness_PasswordReset_Tests.cs`

1. Resumen ejecutivo

Verificar que el flujo de **solicitud de código de recuperación de contraseña** limite el envío a **máximo 5 solicitudes en la última hora** por usuario, y que **no se envíe correo** cuando se excede el límite. Además, validar el comportamiento cuando **el correo no existe**.

Resultado esperado global:

- Se permite el envío si hay < 5 solicitudes en la ventana de 60 minutos.
- Se **rechaza** la solicitud nº 6 en la misma ventana (lanza `BusinessException`), **sin** guardar registro ni enviar correo.
- Si el correo no existe, se lanza excepción informativa y no se realiza ninguna acción posterior.

2. Reglas/Req. de negocio cubiertos

- “Un usuario puede solicitar **hasta 5 códigos** de recuperación en un periodo deslizante de **1 hora**.”
- “Al exceder el límite, el sistema **rechaza** la solicitud y **no envía** correo.”
- “Para correos no registrados, el sistema **no procesa** el flujo de recuperación.”

Fuera de alcance de esta prueba:

- Envío real de email (se usa `IService` moqueado).
- Persistencia real (se moquea `IPasswordReset` y `IUserData`).
- Borde exacto de 60:00 min con reloj controlado (pospuesto a pruebas de integración/ `IClock`).
- Validación y reseteo de contraseña (otros métodos).

3. Entorno y dependencias

- **Entorno:** Local Dev (sin SMTP real).
- **Dependencias moqueadas:**
 - `IUserData` → obtener usuario por email.
 - `IPasswordReset` → conteo y registro de solicitudes.
 - `IService` → captura de intento de envío sin usar SMTP.
 - (Inyectadas pero no relevantes en estos tests: `IRolBusiness` , `IRolUserBusiness` , `IJwtAuthenticationService` , `IPasswordHasher` , `ILogger<UserBusiness>` , `IMapper`).

Artefacto bajo prueba (SUT): `UserBusiness`

Método: `Task RequestPasswordResetAsync(string email)`

Efectos esperados: Guardar registro `PasswordReset` y llamar a `IService.SendEmailAsync` (si no se excede la cuota).

4. Datos de entrada

- `email` : `user@test.com` (usuario existente) y `ghost@test.com` (usuario inexistente).
- Ventana de conteo: `nowUtc.AddHours(-1)` (el método usa `DateTime.UtcNow`).
- Conteos simulados: `0..4` (permitido) y `5` (rechazado).

5. Casos de prueba

CT-01 — Permitido con < 5 solicitudes en 60 min

Precondición: Usuario existente (id=10), conteo en ventana = 0..4 .

Pasos:

1. Arrange: mock de `GetUserByEmailsync` devuelve usuario; `CountRequestsSinceAsync` devuelve $N \in [0,4]$.
2. Act: `RequestPasswordResetAsync(email)` .
3. Assert: no lanza excepción; se invoca `Add(PasswordReset)` y `SendEmailAsync(...)` 1 vez.

Resultado esperado: ✓ Prueba exitosa.

Evidencia:

The screenshot shows the Visual Studio Test Explorer interface. On the left, a tree view lists test items: 'AnprTests (8)', 'UnitTest1 (1)', and 'UserBusiness_PasswordReset_Tests'. Under 'UserBusiness_PasswordReset_Tests', the test 'RequestPasswordReset_Permite...' is selected and highlighted with a red bracket. The table below shows the execution details for this test.

Prueba	Duración	Rasgos	Mensaje de error
AnprTests (8)	1,5 s		
AnprTests (8)	1,5 s		
UnitTest1 (1)	1 ms		
UserBusiness_PasswordReset_Tests	1,5 s		
RequestPasswordReset_Error_Si...	17 ms		
RequestPasswordReset_Permite...	1,5 s		
RequestPasswordReset_Permi...	11 ms		
RequestPasswordReset_Permi...	2 ms		
RequestPasswordReset_Permi...	3 ms		
RequestPasswordReset_Permi...	2 ms		
RequestPasswordReset_Permi...	1,5 s		
RequestPasswordReset_Rechaz...	6 ms		

On the right, the 'Resumen del grupo' (Group Summary) for 'AnprTests.UserBusiness_PasswordReset_Tests.RequestPasswordReset_Permite_Hasta5Menos1EnLaUltimaHora' is shown. It indicates 'Pruebas en grupo: 5' and 'Duración total: 1,5 s'. The 'Salidas' (Outputs) section shows '5 Correcta'.

CT-02 — Rechazado al ser el 6.º intento en 60 min

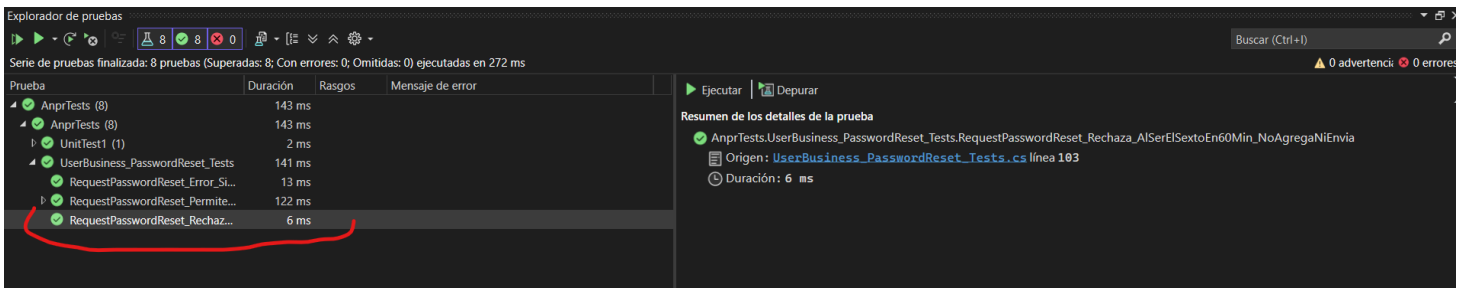
Precondición: Usuario existente (id=20), conteo en ventana = 5 .

Pasos:

1. Arrange: `CountRequestsSinceAsync` → 5; `OldestRequestSinceAsync` devuelve una fecha dentro de la ventana.
2. Act: `RequestPasswordResetAsync(email)` .
3. Assert: lanza `BusinessException` con mensaje que indica límite; **no** se invoca `Add(...)` ni `SendEmailAsync(...)` .

Resultado esperado: ✓ Prueba exitosa.

Evidencia: Excepción capturada; `Times.Never` en `Add / SendEmailAsync` .



CT-03 — Error por correo inexistente

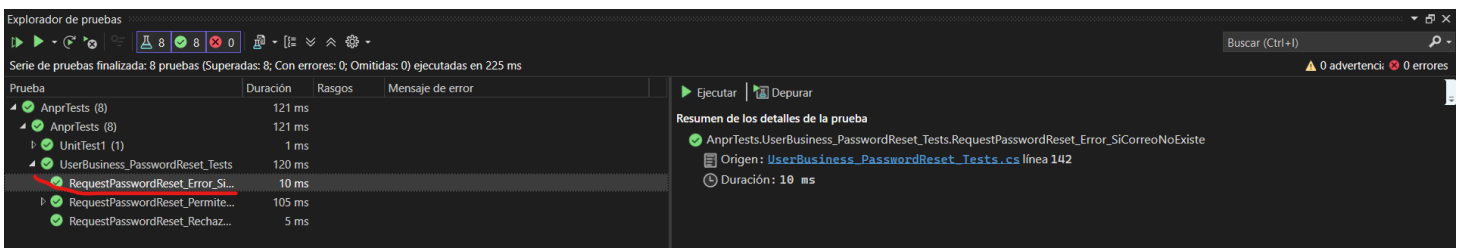
Precondición: `GetUserByEmailsync(email)` devuelve `null` .

Pasos:

1. Arrange: correo no existe.
2. Act: `RequestPasswordResetAsync(email)` .
3. Assert: lanza excepción con mensaje que indica que el usuario no existe; sin llamadas a repositorio ni email.

Resultado esperado: ✓ Prueba exitosa.

Evidencia: `Times.Never` en `Add / SendEmailAsync` y verificación del mensaje.



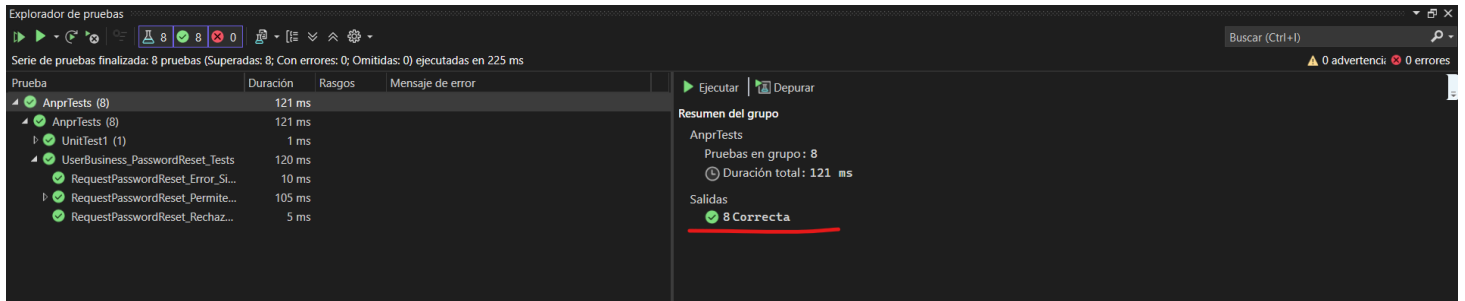
6. Resultado de ejecución

Runner: Test Explorer (Visual Studio).

Resultado: 8/8 pruebas superadas (0 fallidas).

Tiempo total: ~1.6 s.

Evidencia:



7. Criterios de aceptación

- Se comprueba el cumplimiento de los requerimientos mencionados.
- No se envía correo ni se guarda registro cuando se excede el límite.

8. Comentario personal

Todas las pruebas pasaron (8/8). La lógica de límite 5/h se comporta como se esperaba: en el 6.º intento se lanza `BusinessException` y, por eso mismo, el test pasa (el error es el resultado correcto en este caso).